

AUDIT FINAL

ChatCommerce CRM Africa

Analyse complete de securite, architecture, base de donnees et frontend

Projet	ChatCommerce CRM Africa
Technologies	Next.js 16, Prisma, PostgreSQL (Neon), Vercel
Models DB	24 models, 45+ relations, 36 indexes
Routes API	46 endpoints, 9 279 lignes
Components	28 pages, 51 shadcn/ui primitives
Date Audit	14/08/2026
Environnement	Production (Vercel + Neon DB)

TABLE DES MATIERES

1. Note Globale et Scores
2. Architecture et Structure du Projet
3. Securite - Vulnabilites Critiques
4. Securite - Vulnabilites Hautes
5. Securite - Vulnabilites Moyennes et Basses
6. Base de Donnees - Schema et Integrite
7. API - Routes et Logique Metier
8. Frontend - Composants et Experience Utilisateur
9. Performance et Deploiement
10. Plan d'Action Prioritaire
11. Verdict Final

1. Note Globale et Scores

ChatCommerce CRM Africa est une plateforme SaaS multi-tenant destinee aux entreprises camerounaises et africaines. Elle combine la gestion de contacts, leads, commandes, factures, campagnes marketing, et surtout un systeme complet d'agents Telegram bases sur l'IA. L'application est construite avec Next.js 16, Prisma ORM, PostgreSQL heberge sur Neon, et deployee sur Vercel. Malgre une couverture fonctionnelle impressionnante, l'audit revele des vulnerabilites de securite critiques qui doivent etre traitees avant une adoption en production a grande echelle. Le code montre une bonne maitrise de l'ecosysteme React/Next.js et une architecture globalement coherente, mais souffre de decisions de securite temoignaires d'un developpement rapide sans revue formelle. Les sections suivantes detaillent chaque domaine analyse avec des severites attribuees selon l'impact potentiel sur la securite, la stabilite et la maintenabilite du systeme.

4.0 / 10	5.5 / 10	6.0 / 10	7.0 / 10	6.5 / 10
Securite	Architecture	Base de Donnees	Frontend / UX	Performance

NOTE GLOBALE : 5.5 / 10 - Le projet est fonctionnel et impressionnant en termes de couverture. Il necessite des corrections de securite majeures avant une mise en production serieuse.

17	5	5	3	4	5
Critiques	Hautes	Moyennes	Basses	Points Forts	Recommandations

2. Architecture et Structure du Projet

2.1 Vue d'Ensemble Technique

Le projet utilise une architecture monolithique modulaire deployee comme application Next.js serverless sur Vercel. Le choix de Next.js 16 avec le routeur App Router est moderne et performant. L'application fonctionne en mode SPA (Single Page Application) avec un seul fichier page.tsx qui route vers les composants via un store Zustand stockant la page courante. Ce pattern evite les transitions de page completes mais contourne le systeme de routing de Next.js, eliminant les avantages du SSR (Server-Side Rendering) et du code-splitting automatique. La base de donnees PostgreSQL est hebergee sur Neon avec un pooler de connexion, et Prisma ORM gere toutes les interactions avec la base de donnees de maniere type-safe. L'authentification JWT avec la bibliotheque jose (HS256) est correctement implementee en production avec un secret JWT obligatoire. L'architecture multi-tenant est bien respectee : toutes les donnees sont scopees par companyId, avec 24 models Prisma et plus de 45 relations.

2.2 Stack Technologique

Categorie	Technologie	Version	Appreciation
Framework	Next.js	16.1.1	Excellent - Derniere version
Langage	TypeScript	5.x	Bon - Strict partiel (noImplicitAny: false)
ORM	Prisma	6.11.1	Excellent - Type-safe, migrations
Base de donnees	PostgreSQL (Neon)	Serverless	Bon - Pooled connections
UI	shadcn/ui + Radix	Dernier	Excellent - Accessible, composeable
State	Zustand	5.0.6	Bon - Simple, efficace
Style	Tailwind CSS	v4	Excellent - Derniere version
Auth	jose (JWT)	6.2.3	Correct - Mais fallback dev risque
Deploiement	Vercel CLI	59.0.0	Bon - CI/CD automatise
Paieement	Chariow	Integre	Correct - Webhook handle
IA	Mistral / OpenRouter	Integre	Bon - Multi-fournisseur

2.3 Points Forts Architecturaux

- + Multi-tenant bien implemente avec isolation par companyId sur toutes les queries
- + Systeme de plans et limites (starter/pro/business/enterprise) avec enforcement via plan-limits.ts

-
- + Telegram bot system complet : 9 routes API, 12 types d'agents preconfigures, IA integree
 - + Delivery tracking avec gestion de chauffeurs et coordonnees GPS
 - + PWA-ready avec manifest.json et service worker
 - + Dark mode complet via next-themes et variables CSS oklch
 - + Systeme de notifications temps reel via SSE (Server-Sent Events)
 - + Middleware JWT centralise avec bypass pour routes publiques

2.4 Faiblesses Architecturales

- Pattern SPA monolithique : un seul page.tsx pour toute l'app, pas de code-splitting, pas de SSR
- 4 composants megacomposants (> 1300 lignes) dont telegram-page.tsx a 2086 lignes
- Role checks melanges : company_admin = business user MAIS aussi utilise pour les checks admin globaux
- ID admin en dur 'admin-hardcoded-001' present dans 10 fichiers source (front + back)
- Pas de systeme de routing Next.js : impossible de faire des liens directs vers une page
- Pas de tests unitaires, d'integration ou e2e detectes dans le projet
- Variables d'environnement non configurees dans .env local (seulement SQLite fallback)

3. Securite - Vulnabilites Critiques

Les vulnabilites critiques identifiees representent des risques immediats et significatifs pour la securite des donnees des utilisateurs et l'integrite du systeme. Elles doivent etre corrigees en priorite absolue avant toute mise en production a grande echelle. Chaque vulnabilite est documentee avec sa localisation precise, l'impact potentiel, et la correction recommandee.

C1 - Mot de passe admin en dur dans le code source

Le mot de passe de l'administrateur est directement ecrit dans le code source de l'application, accessible a quiconque a acces au depot Git ou au code deploye. La constante `ADMIN_PASSWORD` dans `src/app/api/auth/route.ts` contient la valeur `'Admin@2024'` comme fallback si la variable d'environnement `ADMIN_PASSWORD` n'est pas definie. Le compte admin hardcore avec l'ID `'admin-hardcoded-001'` contourne entierement la base de donnee et obtient automatiquement le role `company_admin` et le plan `enterprise`. Cela signifie que n'importe quel developpeur ou attaquant ayant acces au code source peut se connecter en tant qu'administrateur avec des privileges complets.

Fichier : `src/app/api/auth/route.ts:13` | `src/app/api/auth/change-password/route.ts:31`

La correction consiste a exiger absolument que `ADMIN_PASSWORD` soit defini comme variable d'environnement en production, et a supprimer completement le fallback en dur. De plus, il faudrait envisager de migrer le compte admin hardcore vers la base de donnees pour un controle centralise des credentials, et imposer la double authentication (2FA) pour ce compte.

C2 - Endpoint seed-demo protege par une cle en dur

L'endpoint `/api/seed-demo` est protege uniquement par un header HTTP `'x-admin-key'` avec la valeur en dur `'demo-seed-2024'`. Cette valeur est publiquement devinable et permet a quiconque de decouvrir la route d'injecter des donnees de demonstration dans la base de donnees. Bien que le matcher du middleware bloque techniquement cette route, la cle secrete triviale est un anti-pattern de securite grave. Si le matcher est modifie ou si un bug expose la route, toute la base de donnees peut etre corrompue avec des donnees de test.

Fichier : `src/app/api/seed-demo/route.ts:7`

C3 - Token de bot Telegram expose dans les parametres URL

Le webhook Telegram transmet le token du bot comme parametre de query URL (`?token=XXX`). Ce token apparait dans les logs du serveur, les logs de Vercel, l'historique du navigateur si teste manuellement, les logs CDN/proxy, et les outils de surveillance reseau. Un attaquant qui obtient un de ces logs peut pirater tous les bots Telegram des entreprises clientes. Le token ne devrait jamais etre dans l'URL ou la query string. Telegram supporte le `secret_token` dans `setWebhook` pour valider l'origine des requetes, ce qui eliminerait le besoin de transmettre le token dans l'URL.

Fichier : `src/app/api/telegram/webhook/route.ts:186`

C4 - Route cron automations ouverte en mode developpement

La route `/api/cron/automations` ne verifie aucune authentication lorsque `NODE_ENV` est different de `'production'`. Si le deploiement tourne accidentellement en mode developpement (ce qui est courant avec les deploiements mal configures), n'importe qui peut declencher toutes les automations marketing (campagnes WhatsApp, SMS, emails) sans aucune authentication. Le

CRON_SECRET peut ne pas être défini dans les environnements de staging, rendant la protection inexistante.

Fichier : src/app/api/cron/automations/route.ts:20

4. Securite - Vulnabilites Hautes

Les vulnabilites de severite haute n'ont pas un impact immediat aussi devastateur que les critiques, mais elles exposent le systeme a des risques significatifs qui pourraient etre exploites dans des scenarios d'attaque specifiques.

H1 - Rate limiting en memoire uniquement (inefficace en production)

Tous les rate limiting (login 5 req/60s, inscription 10 req/h, OTP 5 req/h) sont implements en memoire vive avec un dictionnaire JavaScript. Sur Vercel serverless, chaque invocation de fonction tourne dans un processus separe. Un attaquant peut donc brute-forcer les mots de passe avec des tentatives illimitees en production car chaque requete arrive dans une instance differente du serveur. Le code contient d'ailleurs un avertissement : 'DB rate limiting not implemented - using in-memory fallback'. La solution est d'implementer le rate limiting avec Upstash Redis ou Vercel KV qui persistent les compteurs entre les invocations serverless.

Fichier : `src/lib/security.ts:100-146`

H2 - Matcher middleware incomplet pour les routes publiques

Le matcher du middleware utilise le pattern `/api/((?!auth|seed).*)` qui exclut uniquement auth et seed. Les routes publiques telles que `/api/cron`, `/api/chariow/webhook`, `/api/telegram/webhook` sont dans `PUBLIC_PATHS` mais le matcher ne les exclut pas. Chacune de ces routes effectue sa propre verification d'authentification, mais si une route oublie d'appeler `verifyToken()`, il n'y a pas de filet de securite au niveau du middleware. La correction recommandee est de mettre a jour le matcher pour exclure explicitement toutes les routes publiques, ou a l'inverse, de verifier toutes les routes non-exclues par default.

Fichier : `src/middleware.ts:76-81`

H3 - Hachage SHA-256 legacy pour mots de passe

La fonction `verifyPassword` supporte un format legacy SHA-256 avec un simple salt. SHA-256 sans key-stretching est vulnérable aux attaques par force brute GPU (millions de hashes/seconde contre environ 10 000 pour bcrypt). Si des mots de passe utilisateurs sont encore dans ce format, ils sont extremement exposes. Le code supporte ce format pour la migration mais ne force pas la migration automatique vers bcrypt lors d'une connexion reussie avec un hash SHA-256.

Fichier : `src/lib/auth.ts:30-33`

H4 - Token Telegram expose a tous les utilisateurs authentifies

La requete `GET /api/telegram/agents` selectionne le champ token pour tous les utilisateurs. Bien qu'il soit supprime pour les non-admins via un `.map()` manuel, cette approche est fragile. Si le code change ou si une erreur survient avant le stripping, les tokens Telegram complets fuient vers n'importe quel utilisateur y compris les viewers. La correction est de ne jamais selectionner le token dans la requete Prisma pour les non-admins, plutot que de le selectionner puis le supprimer.

Fichier : `src/app/api/telegram/agents/route.ts:28`

H5 - 2FA non enforcee pour le compte admin hardcore

Le login du compte admin hardcore ne verifie ni n'impose la 2FA. Si un utilisateur admin en base de donnees a la 2FA activee, le compte admin-hardcoded-001 peut toujours se connecter avec seulement email + mot de passe, ignorant completement toute protection 2FA. Cela rend la double authentification inutile pour l'administrateur le plus sensible.

Fichier : src/app/api/auth/route.ts:102-122

5. Securite - Vulnabilites Moyennes et Basses

5.1 Vulnabilites Moyennes

M1 - Headers de securite manquants dans le middleware

Le middleware effectue la verification JWT mais n'ajoute aucun header de securite (X-Frame-Options, X-Content-Type-Options, Content-Security-Policy, HSTS, Referrer-Policy, X-XSS-Protection). Ces headers protegent contre le clickjacking, le MIME sniffing et les injections XSS. Note : next.config.ts ajoute certains headers mais pas via le middleware pour les routes API.

M2 - CORS wildcard sur le endpoint SSE

Le endpoint /api/notifications/stream retourne Access-Control-Allow-Origin: * qui permet a n'importe quel site malveillant de se connecter au flux de notifications si un token valide est obtenu via XSS.

M3 - Mots de passe temporaires non transmis par canal securise

L'API de gestion des membres genere des mots de passe temporaires (format CcA-XXXXXXXXXX) mais n'a aucun code pour les envoyer par email. Le mot de passe est stocke (hashe) en base mais jamais communique au nouveau membre.

M4 - Webhook Telegram sans validation de signature

Le endpoint webhook ne verifie pas que la requete provient reellement de Telegram. Il n'y a pas de validation du secret_token. N'importe qui qui decouvre l'URL du webhook peut envoyer des faux messages.

M5 - Cle JWT dev fallback previsible

Si la production tourne accidentellement en mode non-production, tous les JWT peuvent etre forges avec la cle publiquement connue 'chatcommerce-dev-only-fallback-key-2024', accordant un acces complet.

5.2 Vulnabilites Basses

L1 - Pas de protection CSRF

L'API s'appuie uniquement sur les Bearer tokens, ce qui protege naturellement contre CSRF. Cependant, les routes d'authentification acceptent des POST sans token CSRF.

L2 - Codes de backup 2FA non rate-limited

Les endpoints 2FA n'ont pas de rate limiting explicite. Un attaquant avec un token de session valide pourrait brute-forcer le code TOTP a 6 chiffres (1 million de combinaisons) sans etre bloque.

L3 - Logs verbeux exposant des details internes

Des logs console.log dans le code de production exposent des donnees de session, des userId, companyId, et des corps de requetes. Ces logs apparaissent dans Vercel Logs et pourraient fuiter des informations sensibles.

5.3 Tableau Recapitulatif des Vulnerabilites

ID	Severite	Titre	Impact
C1	CRITIQUE	Mot de passe admin en dur	Prise de controle admin totale

C2	CRITIQUE	Seed-demo cle en dur	Injection de donnees demo
C3	CRITIQUE	Token Telegram dans URL	Piratage de tous les bots
C4	CRITIQUE	Cron ouvert en dev	Declenchement arbitraire automations
H1	HAUTE	Rate limiting memoire	Brute-force illimite en prod
H2	HAUTE	Matcher middleware	Routes sans filet de securite
H3	HAUTE	SHA-256 legacy	Cracking GPU des mots de passe
H4	HAUTE	Token Telegram leak	Exposition des tokens bot
H5	HAUTE	2FA admin bypassée	2FA inutile pour admin
M1	MOYENNE	Headers securite	Clickjacking, XSS
M2	MOYENNE	CORS wildcard SSE	Leak notifications via XSS
M3	MOYENNE	Mdp temporaire non envoye	Invitations inutilisables
M4	MOYENNE	Webhook sans signature	Messages Telegram spoofes
M5	MOYENNE	JWT dev fallback	Forging JWT si mode dev
L1	BASSE	Pas de CSRF	Attenuation naturelle par JWT
L2	BASSE	2FA non rate-limited	Brute-force TOTP possible
L3	BASSE	Logs verbeux	Leak info dans logs Vercel

6. Base de Donnees - Schema et Integrite

Le schema Prisma comporte 24 models avec plus de 45 relations explicites et 36 indexes. La conception multi-tenant est globalement bonne avec le modele Company comme hub central. Cependant, plusieurs problemes de conception meritent d'etre releves pour la maintenabilite et l'integrite des donnees a long terme.

6.1 Statistiques du Schema

Metrique	Valeur	Appreciation
Models Prisma	24	Bonne couverture fonctionnelle
Relations explicites	45+	Architecture bien connectee
Indexes @@index	36	Bon pour les performances de lecture
Uniques @@unique	4	Peut etre renforce
Cascade deletes	4	Risque de perte de donnees
Champs JSON	7 types	Pas type-safe, difficile a valider
Champs Float (financier)	8+	Erreur d'arrondi possible
Soft delete	0	Suppression definitive irreversible
updatedAt manquant	5 models	Pas de suivi modification

6.2 Problemes Identifies

! Champs financiers en Float : `Product.price`, `Order.subtotal`, `Order.tax`, `Order.total`, `Invoice.amount`, `Payment.amount` utilisent Float au lieu de Decimal. Cela peut entrainer des erreurs d'arrondi sur les montants financiers. Par exemple, $14900 \text{ FCFA} \times 3 = 44700.0000001$ au lieu de 44700 . Les TODOs dans le schema reconnaissent ce probleme mais la migration n'a pas ete effectuee.

! Pas de soft delete : La suppression de donnees est definitive (hard delete). Pour une application CRM qui gere des donnees client sensibles, il faudrait un pattern de soft delete avec un champ `deletedAt` et des filtres pour exclure les enregistrements supprimes. La perte accidentelle de donnees est irreversible.

! JSON non type : `notificationSettings`, `paymentSettings`, `openHours`, `aiConfig`, `filter`, `targetAudience`, `location` sont stockes comme chaines JSON brutes sans validation de schema. Il n'y a pas de Zod ou de type Prisma pour valider la structure. Cela rend la maintenance difficile et les bugs silencieux possibles.

! Champs `updatedAt` manquants : `Message`, `BusinessService`, `TelegramBooking` et `RatelimitLog` n'ont pas de champ `updatedAt`, rendant impossible le suivi de la derniere modification sans parser les logs ou ajouter des triggers manuels.

! Pas de contraintes de cle étrangere pour la multi-tenancy : Les relations utilisent companyId comme scope mais il n'y a pas de contrainte database-level pour garantir que les enregistrements appartiennent bien a leur company. C'est fait au niveau de l'application (dans chaque query Prisma), ce qui est suffisant mais risque des erreurs si un oublie se produit.

7. API - Routes et Logique Metier

L'API comporte 46 fichiers de route couvrant 14 domaines fonctionnels pour un total d'environ 9 279 lignes de code. La couverture d'authentification est bonne avec `verifyToken` present dans toutes les routes. La logique metier est globalement correcte mais presente des incoherences dans les checks de roles et la gestion des erreurs.

7.1 Couverture par Domaine

Domaine	Routes	Lignes	Complexite
Telegram	9	2 060	Haute - IA, webhooks, bots
Livraisons	5	1 221	Haute - GPS, statuts multiples
Dashboard/Reports/AI	6	1 142	Moyenne - Agregations SQL
Authentification	4	854	Moyenne - JWT, 2FA, OTP
Company/Admin	3	856	Moyenne - Multi-tenant
Campagnes	2	498	Moyenne - Segmentation
Chariow Paiement	2	510	Moyenne - Webhook callback
Paiements Mobile	3	350	Moyenne - Orange/MTN Money
Commandes	1	282	Basse - CRUD standard
Contacts/Leads	3	361	Basse - CRUD standard
Conversations	2	201	Basse - Chat inbox
Produits	2	211	Basse - Catalogue CRUD
Automations/Cron	4	148	Basse - Declencheurs

7.2 Problemes de Logique Metier

! Incoherence des checks de roles : `isAdmin` est defini comme `company_admin || super_admin` dans les routes API, ce qui confond les gestionnaires d'entreprise avec les administrateurs systeme. Un business user `company_admin` a le meme acces qu'un `super_admin` dans la plupart des routes, sauf pour le Global Token. Le ID en dur '`admin-hardcoded-001`' est la seule variable distinction pour les operations systeme.

! Erreurs silencieuses dans le frontend : 8 composants (leads, products, orders, inbox, contacts, automations, drivers, deliveries) avalent les erreurs avec `.catch(console.error)` sans montrer aucun feedback utilisateur. L'utilisateur ne sait pas si l'action a echoue.

! Suppression d'agents reservee aux admins : Les business users `company_admin` peuvent creer des agents mais ne peuvent pas les supprimer via l'API DELETE (403). Le frontend ne montre pas l'erreur car `saveAgent` ne lisait pas le message d'erreur (corrige recemment).

! Services CRUD admin-only : La creation et suppression de services pour un agent Telegram sont reservees aux admins. Les business users ne peuvent pas gerer les services de leurs propres agents.

! Pas de validation Zod pour les inputs API : La plupart des routes utilisent des checks manuels (if !name || !token) au lieu d'un schema Zod centralise. Cela rend la validation incomplete et difficile a maintenir.

8. Frontend - Composants et Experience Utilisateur

Le frontend est construit avec 28 composants pages et 51 primitives shadcn/ui pour un total d'environ 14 500 lignes de code applicatif. L'experience utilisateur est globalement bonne avec un dark mode complet, un design responsive sur 25/28 pages, et des indicateurs de chargement sur 20/21 pages. Cependant, plusieurs problemes significatifs affectent la qualite, la maintenabilite et l'accessibilite.

8.1 Megacomposants - Probleme Principal

Quatre composants dépassent 1 300 lignes, ce qui constitue un anti-pattern majeur de maintenance. Le plus gros, telegram-page.tsx avec 2 086 lignes, contient environ 20 hooks useState, 15 fonctions callback, 4 dialogues inline, 2 onglets, et tout le CRUD pour agents, services, reservations, config IA, token global, et webhook. Ce composant devrait etre decompose en au moins 6 sous-composants : AgentList, AgentForm, AgentConfigDialog, BookingManager, ServiceManager, et GlobalTokenCard. La meme recommandation s'applique a settings-page.tsx (1 791 lignes), auth-page.tsx (1 472 lignes qui combine page marketing, login et inscription), et admin-page.tsx (1 347 lignes).

8.2 Erreurs Silencieuses

Huit composants avalent les erreurs API sans aucun feedback utilisateur. Les fetch echouent silencieusement et seul console.error est appele. Les pages affectees sont : leads-page, products-page, orders-page, inbox-page, contacts-page, automations-page, drivers-page et deliveries-page. L'utilisateur ne voit ni toast, ni message, ni indicateur visuel que quelque chose a echoue. La correction est d'ajouter des toasts.error dans chaque catch.

8.3 Accessibilite

L'accessibilite est quasi inexistante. Seuls 2 fichiers (sidebar et notification-bell) ont des attributs aria-label et aria-expanded. La majorite des boutons icon-only n'ont pas de labels accessibles. Les elements interactifs custom comme les cartes d'agent utilisent div onClick au lieu de button. Il n'y a pas de gestion du focus apres ouverture ou fermeture de dialogues, pas de regions aria-live pour le contenu dynamique, et seulement 1 utilisation de sr-only. Le site ne serait pas conforme WCAG 2.1 AA.

8.4 Incoherences Visuelles

La page Telegram est la seule page qui n'utilise pas le composant Header partage, ce qui signifie pas de toggle de theme, pas de recherche, pas de notifications. La page auth combine trois vues distinctes (marketing, login, inscription) dans un seul composant de 1 472 lignes. products-page.tsx utilise des balises img brutes au lieu du composant Image de Next.js, ignorant l'optimisation d'images. Les boites de dialogue natives confirm() sont utilisees dans 4 composants et ne respectent pas le dark mode.

9. Performance et Deploiement

9.1 Performance

- > Build reussi en 50 secondes sur Vercel (Turbopack + cache). Taille de upload 706.7 KB. Pas de fichiers statiques volumineux. Le lazy loading des pages n'est pas utilise car c'est une SPA, ce qui signifie que tout le JavaScript des 28 pages est charge des le premier hit. Un decoupage par route avec Next.js dynamic imports reduirait significativement le temps de chargement initial.
- > La base de donnees Neon avec pooler de connexion est bien configuree pour le serverless. Cependant, il n'y a pas de mise en cache des requetes frequentes (dashboard stats, plan limits). Chaque page load declenche plusieurs requetes database qui pourraient etre mise en cache avec React Query ou un cache LRU.
- > Le Telegram webhook handler effectue jusqu'a 4 requetes database par message recu (find agent, find services, create booking, update stats). Pour un bot populaire, cela pourrait devenir un goulot d'etranglement. L'ajout d'un cache en memoire pour les configs d'agent serait benefique.
- > Pas de monitoring ou d>alerting configure. Il n'y a pas de Sentry, DataDog, ou equivalent pour tracer les erreurs en production. Les erreurs sont seulement visibles dans les logs Vercel qui ont une retention limitee.

9.2 Deploiement

- ! Le deploiement est fait via Vercel CLI avec un token personnel encode en dur dans la commande de deploiement. Le token Vercel 'vcp_3LtT8HA9eBHyyxkeSLIiUrEqitxuYL4R2QVUABAQFP9mv23LPt26Tggy' est visible dans l'historique des commandes et les logs. Il devrait etre stocke dans les secrets du CI/CD ou dans un fichier .env non versionne.
- ! Le fichier .env local pointe vers SQLite (file:./db/custom.db) au lieu de PostgreSQL Neon, ce qui signifie que le developpement local ne reflète pas l'environnement de production. JWT_SECRET et toutes les autres variables d'environnement ne sont pas configurees, forçant le fallback en dur.
- ! Le middleware genere un avertissement : 'The middleware file convention is deprecated. Please use proxy instead.' La migration vers le nouveau systeme proxy de Next.js 16 n'a pas ete effectuee et constitue une dette technique.

9.3 Dependances

Le projet utilise 37 dependances de production et 12 de dev. La plupart sont a jour. Cependant, next-auth est installee mais non utilisee (auth JWT custom est a la place). @mdxeditor/editor et react-syntax-highlighter sont installees mais leur utilisation n'est pas claire. ws (WebSocket) est installe pour le mini-service Telegram mais pas pour l'app principale. Un audit des dependances inutilisees reduirait la taille du build.

10. Plan d'Action Prioritaire

Ce plan d'action priorise les corrections par urgence et impact. Les actions critiques doivent être menées immédiatement, suivies des actions à court terme (1-2 semaines), moyen terme (1-2 mois) et long terme (3-6 mois). Chaque action est estimée en effort pour aider à la planification.

Phase 1 : Corrections Immédiates (0-3 jours)

Action	Effort	Priorité
Supprimer le fallback du mot de passe admin en dur	4h	CRI TIQ UE
Implementer le rate limiting avec Upstash Redis	8h	CRI TIQ UE
Ajouter les headers de sécurité dans le middleware	2h	HA UT E
Migrer le token Telegram du query param vers un header	4h	CRI TIQ UE
Corriger les 8 composants avec erreurs silencieuses	4h	HA UT E
Enlever la vérification dev-only dans les crons	1h	CRI TIQ UE

Phase 2 : Court Terme (1-2 semaines)

Action	Effort	Priorité
Migrer les Float financiers vers Decimal dans Prisma	12h	HA UT E
Migrer les mots de passe SHA-256 vers bcrypt	6h	HA UT E
Ajouter la validation de signature Telegram webhook	3h	MO YE NN E

De-composer telegram-page.tsx en sous-composants	16h	HAUT E
Implementer l'envoi d'emails pour mots de passe temporaires	8h	MOYENNE
Ajouter un monitoring d'erreurs (Sentry ou equivalent)	4h	MOYENNE
Migrer le middleware vers le nouveau systeme proxy	6h	MOYENNE

Phase 3 : Moyen et Long Terme (1-6 mois)

Action	Effort	Priorite
Implementer le soft delete pattern sur les models cles	16h	MOYENNE
Ajouter la validation Zod sur toutes les inputs API	20h	MOYENNE
Migrer vers Next.js file-system routing (SSR)	40h	BASSE
Ajouter des tests unitaires et d'integration	60h	BASSE
Ameliorer l'accessibilite (WCAG 2.1 AA)	30h	BASSE
Implementer la mise en cache des requetes frequentes	12h	BASSE
Nettoyer les dependances inutilisees (next-auth, ws, etc.)	2h	BASSE

11. Verdict Final

ChatCommerce CRM Africa est un projet ambitieux et fonctionnellement riche. La couverture des fonctionnalités CRM est impressionnante pour un projet de cette envergure : gestion de contacts, leads, conversations WhatsApp, produits, commandes, factures, campagnes marketing, automatisations, livraisons avec suivi GPS, et surtout un système complet de 12 agents Telegram avec IA intégrée. L'architecture technique est moderne et globalement cohérente, avec un bon choix de stack (Next.js 16, Prisma, PostgreSQL, shadcn/ui, Tailwind v4). Le système de multi-tenancy est bien implémenté et les 4 plans de tarification sont correctement gérés.

CEPENDANT, le projet souffre de vulnérabilités de sécurité critiques qui rendent le déploiement en production dangereux sans corrections préalables. Le mot de passe admin en dur, le rate limiting inefficace, le token Telegram dans les URLs et les clés secrètes en dur sont des failles qui pourraient être exploitées rapidement par un attaquant motivé. La confiance excessive dans les checks côté client (role depuis localStorage) et la confusion entre company_admin (business user) et super_admin (system admin) créent des risques de privilège escalation.

Le verdict est nuancé. Sur le plan fonctionnel, le projet mérite un **7/10** pour sa couverture et son ambition. Sur le plan sécurité, il mérite un **4/10** en l'état. Avec les corrections de la Phase 1 (estimées à 23 heures de travail), la note de sécurité pourrait monter à **7/10**. La note globale pondérée est de **5.5 / 10**.

VERDICT : 5.5 / 10

FONCTIONNELLEMENT IMPRESSIONNANT - La couverture CRM et Telegram est remarquable.

SECURITE INSUFFISANTE - 4 vulnérabilités critiques doivent être corrigées immédiatement.

MAINTENABILITE A AMELIORER - Megacomposants, erreurs silencieuses, accessibilité.

POTENTIEL ELEVE - Avec les corrections recommandées, ce CRM peut être production-ready.

Recommandation finale : Corriger les 6 actions de la Phase 1 immédiatement (estime : 23h). Poursuivre avec les actions de Phase 2 dans les 2 semaines suivantes. Le projet a un potentiel réel et une couverture fonctionnelle qui le distingue. Avec les corrections de sécurité et de maintenabilité recommandées, ChatCommerce CRM Africa peut devenir une plateforme CRM compétitive pour le marché africain.